



山东大学
SHANDONG UNIVERSITY

SOAR 2026 MiniCPM-SALA 推理优化技术复盘

教学复盘：模型特点 · 方法空间 · 决策依据 · 失败复盘

SOAR 2026 参赛复盘

MiniCPM-SALA / SGLang / W4A16 / FP8KV

2026-06-08



一、赛题与目标：先搞清楚“在比什么”	1
二、先把模型讲透：SALA 为何不是普通 Transformer	4
三、量化：为什么能压，以及压什么	8
四、量化范围：不是“要不要 W4”，而是“哪里能 W4”	12
五、工程边界：artifact 与 runtime 决定早期成败	15
六、KV cache：FP8KV 高潜力，但当前没打对	18
七、性能旋钮与验证体系	21

01 一、赛题与目标：先搞清楚“在比什么”



赛题：固定模型 + 固定单卡 + 隐藏长上下文

我们能改的只有“推理栈”

赛题对象是 OpenBMB 的 **MiniCPM-SALA**(9B 参数 / 1M 长上下文)。参赛者**不能改模型能力**，只能在一块固定的 **RTX 6000D(84GB)** 上，用量化 / KV cache / prefill / kernel fusion 等系统手段把推理跑得更快。

所以这道题不是“挑一个最快的量化算法”，而是——**在不掉准确率的前提下提速。**

维度	本赛设定
模型	MiniCPM-SALA 9B / 1M(不可改)
硬件	单卡 RTX 6000D 84GB
评测集	隐藏：长文 QA / NIAH(大海捞针) / MCQ(选择题)
打分	先正确性 gate，再看性能
可动	量化 / KV / prefill / fusion

两段式打分

- 1. **正确性 gate**: 准确率不过门槛，性能再快也判 0 分。
- 2. **性能**: 过 gate 后比三档时长 S_1 / S_8 / S_{max} (并发 1 / 8 / 最大，越小越快)。
 - 比较实验只看**原始正确率**(与本地可比)，别被平台加权分误导。
 - gate 工作假设~80%，真实阈值未公布。

路线	思路	现状
BF16 安全网	不量化，仅调 prefill / 显存	保底可跑，作为“地板”
W4A16 主线	权重压 4-bit 提带宽	已稳定过 gate 且超过 BF16，当前最佳

锚点 已有一个“安全基线”成绩：新变体须**明显高于它**，才值得占用一个 5h 平台 slot。

目标 对齐冠军队大部分技术、冲击更高排名；主线已领先 BF16，但距目标仍有缺口。

02 二、先把模型讲透：SALA 为何不是普通 Transformer

SALA 是一个“混合注意力长上下文 runtime”

和普通 Transformer 的根本不同

普通 Transformer: **每层都是同一种 full attention**, 所有历史 token 的 K/V 进同一个不断增长的 KV cache。

SALA: **不同层换不同机制**。源码里只有**两类 mixer**——minicpm4(标准 GQA: 多 query 头共享一组 K/V)与 lightning(线性递归)。**正是这套混合机制, 让 9B 模型扛得住 1M 上下文。**

项	值(据项目文档)
参数量	≈9B
上下文	1M(config.json 不在本仓库)
层类型	minicpm4 / lightning
运行时行为	full / sparse / lightning
缩放	scale_emb / scale_depth / scale_width

注 真实层数 / 维度由模型自带 config.json 决定, 仓库内无该文件, 数字以文档为准。

关键 minicpm4 层运行时还会按序列长度**自动在 dense 与 sparse 两条路径间切换**——所以运行时表现为 full / sparse / lightning **三种行为**, 但**不是**三种独立层类型。

先立框架：层只有两类(minicpm4 / lightning)；但 minicpm4 会按序列长度切换 dense / sparse，所以运行时看到三种行为。

行为	类比	怎么存“记忆”	优化含义
Full / 标准	每问一词就把整本书重读一遍	全量 KV cache，随长度线性增长	精度基准；KV 是带宽 / 显存大头
Sparse(InfLLM-v2)	先做摘要卡片，只翻最相关几页	仍存 KV，但 top-k 只读一小部分块	长上下文核心路径；依赖定制 CUDA kernel，硬假设 bf16
Lightning(线性)	不留全书，只滚动更新一份“笔记”	固定大小递归 state(走 mamba 状态池)	state 不随长度增长 ——SALA 扛 1M 的关键；不走 KV cache，KV 量化对它无效

- **Sparse 不是独立层**：它是 minicpm4 层在 $seq_len \geq dense_len$ 时走的运行时路径(--dense-as-sparse 让所有 batch 都走稀疏)；两级 key 压缩做“摘要”，再选 top-k 块做 full attention。
- **Lightning 数学上 $\neq$ KV cache**：它是线性递归(SimpleGLA / Gated Delta Rule，用 ALiBi 斜率做衰减)，量化噪声沿**时间步**累积，是长上下文下的精度敏感区。

SALA 专有缩放(源码核实)

- 每子层残差: $\text{residual} + \text{hidden} * (\text{scale_depth} / \sqrt{L})$, L =层数(本模型 ≈ 32), 这个系数**很小**(约 0.2)。
- 词嵌入 $\times \text{scale_emb}$ (本模型=12, 放大后激活幅度大 $\rightarrow$ 易出 outlier、FP8 易饱和); 出 logits 前 $\div \text{scale_width}$ 。
- RoPE 在 fp32 内计算再 cast 回。

记住 每子层“一点点”乘性误差, 沿几十层累积会放大成重复输出——这正是后面 fusion 翻车的根因。

结构决定优化边界

敏感、别乱动:

- lightning 递归(噪声沿时间步累积)
- sparse kernel(硬假设 bf16, 改 dtype 易在 kernel 边界 crash)
- scale_depth 残差语义

可压、收益大:

- full / sparse 层的 KV cache(FP8-KV 主战场)
- 权重 GEMM(W4A16 带宽收益)

03 三、量化：为什么能压，以及压什么

decode 是“访存受限”的

每生成一个 token, 都要把**全部权重**从显存搬一遍——瓶颈在**带宽**不在算力。

把权重从 16-bit 压到 4-bit, 模型体积**约压到 1/4**(实测), 理论上接近 **4× 带宽节省**。

量化的几个维度

- **压什么**: weight-only W4A16 / weight+act W8A8 / FP8。
- **粒度**: per-tensor / per-channel / **per-group**。
- **对称 vs 非对称**。

注 “4× 带宽”是冠军队(智算一队)笔记的论断, 本队未独立测量; 本队实测到的是 GPTQ MLP-only 路线**端到端时延(并发 1)约降三成**。

硬约束 SGLang gptq_marlin(4-bit 权重 GEMM kernel)只认 bits=4, sym=True(uint4b8 打包), 且缩放 $scale = \max(abs(w)) / 7$ (**不是** /8, 否则正向 outlier 幅度损失约 1/8)。

四种量化算法：方法空间一览



方法	基本思路	优点	缺点 / 风险	本比赛选择
RTN	直接四舍五入到 4-bit 网络，无校准、无误差补偿	实现极简、无需校准、产物可控、跑得快	4-bit 误差无补偿，对 SALA 质量明显不足	仅作格式 / 链路诊断，非主线
GPTQ	用 Hessian 逐层把量化误差补偿到未量化权重	质量潜力高；SGLang 原生 gptq_marlin 路径	与 SALA 自定义 loader / 环境兼容复杂	主线 ；修复后稳定过 gate
AWQ	按激活幅度找缩放，先保护显著 outlier 通道再量化	理论契合 SALA 激活 - outlier(scale_emb/scale_depth 偏大)	走 llm-compressor，另一条工程链	高潜力 备选 ，未上平台验证
混合精度	敏感模块留 BF16，其余 W4A16	用少量高精度层换回 acc，针对性强	量化端与加载端 skip 规则须逐字对齐	MLP-only 即实际过 gate 配置

算法	本赛实测	结论
RTN	质量明显不足(远低于门槛)	只能做格式 / 链路诊断
GPTQ	修复后稳定过 gate	本赛主线
AWQ	仅打包, 未上平台	潜力大但未验证

注 AWQ 仅有预估、未上平台，不宜与主线直接比较。

选 GPTQ 的真正理由

不是因为它最简单，而是因为它的**每个失败都能单变量定位与修复**：

- qzeros: $0x77 \rightarrow 0x88$
- tokenizer 被重写 $\rightarrow$ 强制覆盖回 base
- 版本钉: `transformers == 4.57.1`

逐个 hard constraint 复现 / 修复，最终从 `acc=0` 走到稳定过 gate。

04 四、量化范围：不是“要不要 W4”，而是“哪里能 W4”

方法	基本思路	优点	缺点 / 风险	本比赛选择
MLP-only W4	只量化 MLP， attention 全保 BF16	风险最低、变量最少；避开已知敏感的 attention	拿不到 attention GEMM 的加速	最先稳定过 gate ，质量 anchor
Full W4A16	MLP + q/k/v/o 全量化	S1 明显更快(attention-GEMM 加速真实)	acc 掉 2-4pp，易跌破门槛	可跑但 trade 不优
Selective BF16	只把敏感模块恢复 BF16	可在 acc-速度边界精细调	组合爆炸，需单变量队列 + artifact surgery	后期最贴 SALA 的主方向
Lightning-skip	把 lightning 层 MLP 回退 BF16	针对递归长程误差累积假设	需物理重写 shard，否则 loader KeyError	思路合理，实现未过平台

注 “MLP 最耐量化”是通用经验；本赛只直接验证了 **attention 投影敏感**，故“保 attention BF16”。

变体	正确率	解码速度	最终分
MLP-only(anchor)	√ 过 gate	基准	基准
1. 更大 prefill	≈ 基准	≈ 基准	≈ 基准
Full W4(含注意力)	↓ 掉 2-4pp	↑ 快约 13-14%	≈ 持平 / 略降

读数 full W4 的解码确实更快(attention-GEMM 加速真实), 但正确率掉一截, 最终分被抵消。

关键直觉(记结论不记数字)

- MLP-only 的量化代价**很小**(相对 BF16 不到 1pp), 且稳稳高于门槛。
- Full W4 多换来的速度, 被多掉的正确率吃掉。

结论 加速 ≠ 涨分：本赛打分里准确率权重大到能把延迟收益吃光, 所以 MLP-only 这种混合精度才是更优 trade。

05 五、工程边界：artifact 与 runtime 决定早期成败

一条极易错的链路

submission.tar.gz → prepare_env → prepare_model
→ 量化产物 → SGLang loader → MiniCPM backend →
CUDA kernel

任何一层语义对不齐，都表现成“模型坏了”。 同一份
W4A16 权重，只是换个 runtime dtype，就能从 startup
crash 变成稳定过 gate。

runtime dtype	结果
fp16(sed-patch / --dtype float16)	平台 CUDA-graph crash: query / key dtype 不一致 → 此路不通
bf16 (--dtype bfloat16)	稳定过 gate, W4A16 成功拐点

注 bf16 能成，靠 SGLang 把 Marlin 的 fp16 输出 cast 回 bf16
——这是**实测推断**的行为，非文档保证。

边界	表面现象	真实根因	修复
qzeros	acc=0	写 0x77, Marlin 要 0x88(每权重 +1 偏置)	后处理改写(幂等)
Tokenizer	本地对 / 平台乱码	save() 重写模板, 旧 transformers 忽略 sidecar	强制覆盖回 base
Config	model_type / 属性错误	auto_map.AutoConfig 抢占; 派生只读属性被序列化	删除二者(v3/v5/v5c 三连败证实)
Dynamic skip	KeyError	规则没匹配 fused gate_up_proj; 残留 orphan 量化张量	按 SGLang 名写 + 物理清张量
版本 / 网络	PREPARING 卡死	版本不一致; 平台连不上 GitHub	钉 transformers==4.57.1 + 离线 wheel

教训 先用 preflight 把这些语义边界**机械化锁死**, 再谈量化质量——**本地通过 ≠ 平台通过**。

06 六、KV cache: FP8KV 高潜力，但当前没打对

KV cache(缓存历史 K/V 避免重算)随序列长度**线性膨胀**，是显存容量 + 带宽的双重瓶颈：decode 每生成一个 token 都要把整段 KV 读一遍。压缩它有几条路——

方法	基本思路	优点	缺点 / 风险	本比赛选择
BF16 KV(基线)	不量化, backend 直接读写	数值无损、完全兼容	占满显存、解码受带宽限制	所有过线提交的实际配置
FP8 e4m3	KV 存 8-bit(精度高、范围小 max=448)	容量 / 带宽各省一半; 尾数多一位更准	范围小, 易被 scale_emb 撑爆饱和	本赛主选格式 + 逐层标定
FP8 e5m2	改用大范围(±57344)	几乎不因离群值饱和	比 e4m3 更糙	仅作范围备份
scale 粒度	scalar(每层一标) → per-head / per-group	越细越贴 GQA / head 分布	scalar 太粗(默认 1.0 直接复读崩); 细粒度需 runtime bridge 易错	scalar 不够 → 后续做 per-head

注 只有 full / sparse 层有 KV; **lightning 层走递归 state, 没有 KV**——FP8-KV 只省一部分层。

两道关：先“跑起来”，再“过精度”

第一关·工程(已过)

旧结论“sparse backend 拒绝 FP8”被自己的实验**推翻**：真因是 SALA 把 KV 的 fp8 dtype **传染给了 Q**，触发 FlashInfer FA2 的 fp8 断言(该断言在 kernel 编译阶段就触发，与显卡型号无关)。

Path Y：Q 保持 bf16、KV pool 仍存 fp8、读取时反量化回 bf16，server 起得来、长 prompt 通过。

第二关·精度(未过)

默认 $\text{scale}=1.0 \times \text{scale_emb}$ 偏大 $\rightarrow$ FP8 饱和 $\rightarrow$ 长输出复读崩溃。标定路线(逐层标定 / dense 层 q.k.v 回 BF16 / 重载后标定)逐步逼近，但最好的变体**仍低于安全锚点**，没打赢 BF16 路线。

硬件 Blackwell 这代卡上两个 backend 都拒绝真正的 FP8 注意力计算 $\rightarrow$ FP8-KV 当前**结构性受限**，省下的显存还没换成吞吐。

07 七、性能旋钮与验证体系



只动调度 / 显存(基本不掉精度)

- **Chunked prefill**: 把长 prompt 切块送。安全配置稳定过 gate; 块开太大 + 显存留太满 → **中途 OOM**, 分数几乎归零。
- **--dense-as-sparse**: 所有 batch 走稀疏路径(baseline 默认)。

动数值计算图(会悄悄改输出)

Kernel fusion:

- 安全子集: 仅去 RoPE 的冗余 fp32 upcast(仍需长 canary 验)。
- 危险: fused add_rmsnorm 把 scale_depth 残差改成 Llama 式, 误差沿全部子层累积。

注 “更大块更快”只是**事前预测**; 平台实测与小块**基本持平**。

反例 opfusion v1 平台正确率**断崖式塌方**(过半样本崩坏)、最终分归零, 而三档速度几乎不变——服务器健康, 纯精度塌方。

四层验证：每层证明什么、不能证明什么



验证层	能证明	不能证明 / 教训
3-prompt smoke	服务起得来、长 prompt 不崩	不能证明质量
30 条本地	快速发现明显崩坏	对 per-token 漂移 假阳性 (opfusion 即栽在此)
≥100 长上下文 canary	暴露长输出数值漂移累积	仍不等于隐藏集
平台 150 样本	唯一权威分数	5h / 次，不能盲试

核心教训 **smoke 通过 ≠ 平台不掉分**：opfusion 本地 30 条看着没问题，平台却塌方。凡触及 RoPE / RMSNorm / residual / scale 的改动，必须用 ≥100 样本含长上下文(≥16K)验证。

SALA 优化的难点不是“选一个量化算法”，
而是让 [量化精度](#) 同时与模型结
构对齐。



山东大学

SHANDONG UNIVERSITY

感谢聆听

Thanks for Listening